

GrabOn AI Labs

Agentic AI Engineer Challenge

6 Assignments. Choose One. Ship It.

Format	6 independent assignments. Pick one.
Difficulty	Hard. 3 to 4 days of focused work.
Deadline	3 to 4 days from the date you receive this document.
Language	Python or TypeScript. Your choice. Either is fine.
Frameworks	Use whatever you want. LangChain, CrewAI, AutoGen, raw API calls. But you must be able to explain what is happening under every abstraction.
LLM Providers	Minimum 2 real providers with live API calls. All major providers have free tiers. Details in Resources section.
What This Tests	Agent loops, multi-LLM orchestration, eval engineering, tool-calling, failure recovery, production instincts.
What This Does NOT Test	UI polish, product design, frontend skill, business strategy.
Submission	GitHub repo + eval report + 15 min Loom video.
Contact	careers@grabon.in with subject: AI Labs - [Your Name]

The Six Assignments

Each assignment tests one distinct axis of the Agentic AI Engineer role. No two overlap. Together, they cover the full scope of what AI Labs ships. Pick the one closest to your strength.

#	Name	Core Skill Tested	Not Tested Here
01	The Orchestrator	Multi-LLM routing with cost, latency, accuracy SLAs	Agent autonomy, process replacement
02	The Loop	Single-agent autonomy: plan, act, observe, decide, recover	Multi-model routing, multi-agent coordination
03	The Guard	Eval engineering: regression detection, statistical testing, CI/CD	Agent execution, tool-calling

04	The Replacer	Workflow automation: replace a real process, measure ROI	Eval pipelines, code generation
05	The Coder	Code generation with RAG, self-testing, iterative repair	Process automation, multi-agent
06	The Swarm	Multi-agent coordination: shared state, conflict resolution	Single-agent loops, eval infrastructure

Before You Start

Read this section completely. It answers the questions you are about to ask.

Timeline

- You have **3 to 4 days** from the date you receive this document. If you received it on Monday morning, your submission is due by Thursday end-of-day.
- If you need an extension for a legitimate reason (travel, health, notice period), email us before the deadline. We will almost always say yes. What we will not do is grant extensions after the deadline has passed.
- We evaluate the output, not the hours. Some candidates finish in 2 days. Some use all 4. Both are fine.

Process After Submission

- **Day 1-2:** We clone your repo, run your agent, run your eval, and watch your Loom. We read your README before running anything.
- **Day 3-4:** If your submission clears the bar, we schedule a 60-minute technical deep-dive. You will walk us through your architecture, we will stress-test your agent live, and we will ask you to make a change on the spot (a prompt swap, a model swap, a new eval case). This is not a gotcha. It tests whether you actually built it and understand it.
- **Day 5:** Decision. You will hear back within 5 business days of submission regardless of outcome. If we reject, we will tell you why.

Language, Frameworks & Tools

- **Language:** Python or TypeScript. Pick one. Both are first-class in our stack.
- **Frameworks:** You may use LangChain, LangGraph, CrewAI, AutoGen, Haystack, LlamaIndex, Instructor, Pydantic AI, or any other framework. You may also use raw API calls with no framework. We do not care what you use. We care that you can explain what every layer of your stack is doing. If you use LangChain and cannot explain what happens between your function call and the HTTP request to Anthropic's API, that is a problem.
- **Coding tools:** You are expected to use Claude Code, Codex, Cursor, or similar AI coding tools. This is an AI engineering role. Using AI to write code is the job, not a shortcut. But: the code must be reviewed, coherent, and free of hallucinated imports or dead functions. AI-generated slop is obvious and will be penalized.
- **Infrastructure:** Docker is welcome but not required. If you use Docker, include a working docker-compose.yml. If you don't, your setup instructions must work on a clean Ubuntu or macOS machine. We must be able to go from git clone to a running agent in under 15 minutes.
- **Frontend:** This is not a frontend role. If your assignment needs a dashboard or UI, a terminal interface, a simple HTML page, or a Streamlit app are all fine. A terminal that clearly shows agent state beats a polished React app that hides details. Do not spend your time on CSS.

What We Mean by 'Minimum Bar'

Each assignment has a Minimum Bar section. This is the line between 'evaluated' and 'auto-rejected.' If your submission does not meet every item in the Minimum Bar, we will not review it further. Everything above the Minimum Bar earns you a higher score. Think of it as: the Minimum Bar gets you into the room. The full Technical Requirements section is what gets you the offer.

Assignment 01: The Orchestrator

Multi-LLM Task Router for GrabOn's Agent Workloads

Build the production orchestration layer that routes GrabOn's real AI workloads across Claude, GPT, Gemini, and a local model, with explicit cost, latency, and accuracy SLAs per task type

[\[Multi-LLM\]](#) [\[Orchestration\]](#) [\[Cost SLA\]](#) [\[Latency SLA\]](#) [\[Fallback Chains\]](#) [\[Shadow Testing\]](#) [\[Eval\]](#)

Difficulty

Hard. 3 to 4 days.

What This Tests

Multi-model fluency, cost engineering, routing policy design, eval-driven improvement

The Problem

GrabOn processes 96M+ transactions per year across 3,500+ merchants. AI Labs is building agents for every layer: deal freshness verification, GrabInsurance intent classification, GrabCredit underwriting narratives, multi-channel deal copy production, Rakuten attribution analytics. Each workload has radically different model requirements. GrabInsurance needs sub-500ms intent classification at checkout. GrabCredit narratives go to Poonawalla Fincorp's compliance team and must be flawless. Deal copy for 21,000 merchants needs to be good-enough at massive scale. Telugu localization requires cultural nuance that most models get wrong.

At GrabOn's scale, the wrong model on the wrong task either costs more than the engineering team's salaries or produces garbage that reaches a partner or 40M subscribers. Build the router that makes the right tradeoff for every task automatically, and proves it with data.

Technical Requirements

- **Define 6 GrabOn-specific task types** with different cost/latency/accuracy profiles: (1) Merchant deal extraction from raw HTML, (2) Insurance intent classification from deal objects, (3) Credit narrative generation from transaction data, (4) Deal copy generation for channels (email, WhatsApp, push), (5) Attribution analysis from click/conversion events, (6) Hindi/Telugu cultural localization of deal copy. Each must have a documented SLA: target p95 latency, max cost per call, min accuracy threshold.
- **Support at least 4 LLM providers** with real API calls: Claude (Sonnet + Haiku), GPT (4o + 4o-mini), Gemini (Flash + Pro), and one local/open model via Ollama. The router must make genuinely different selections for different task types.
- **3 routing strategies** as pluggable policies (not if-else chains): (a) Cost-minimizing for high-volume deal copy, (b) Latency-minimizing for GrabInsurance checkout, (c) Quality-maximizing for GrabCredit narratives to Poonawalla. Switchable at runtime.
- **Fallback chain with budget enforcement.** On failure (timeout, rate limit, malformed output), retry with next-best model. Exponential backoff with jitter. Per-task budget ceiling. A single deal copy task must never exceed Rs. 2, even with retries. Log every fallback event.

- **Eval harness with 50+ test cases** across all 6 task types using realistic GrabOn data (real merchant categories, real deal formats). Ground-truth outputs, automated scoring. Runs end-to-end in under 10 minutes.
- **Shadow testing mode.** Send same task to 2+ models, score both, log which won. After 50+ comparisons, produce a routing policy update recommendation with cost/accuracy tradeoff.
- **Real-time dashboard.** Cost per task type, p50/p95/p99 latency per model, accuracy trend, fallback rate, total spend. Must answer: 'What does it cost to process 3,500 merchants per week?'

Minimum Bar (below this = auto-reject)

- At least 2 LLM providers making real API calls (not mocked)
- Router makes different model selections for different task types (not everything on one model)
- Eval harness exists, runs, and produces a pass/fail report
- Fallback logic handles at least one real failure scenario (timeout or rate limit)
- README documents the routing rationale per task type

What Will Be Stress-Tested in the Deep-Dive

- We will kill a model endpoint mid-stream. Does the fallback chain catch it?
- We will change the budget ceiling to Rs. 0.01/task. Does the router switch models?
- We will ask you to add a 7th task type live. Can you do it in under 10 minutes?
- We will ask: 'At 96M tasks/year, what does this routing policy cost?' Is the answer in your README?

Assignment 02: The Loop

Autonomous Agent That Runs GrabOn's Backend at 3am

Build an agent that takes a complex, multi-step GrabOn operational task, plans its approach, calls tools, observes results, decides what to do next, recovers from failures, respects budget and time limits, and produces a verifiable outcome without human intervention

[Agent Loop] [Plan-Act-Observe-Decide] [Tool-Calling] [Failure Recovery] [Budget Enforcement] [Observability]

Difficulty**Hard. 3 to 4 days.****What This Tests****Agent architecture, tool design, failure recovery, observability, production instincts**

The Problem

GrabOn's production environment is messy: Myntra changes its HTML overnight, Zomato's deal page sits behind Cloudflare, PayU's sandbox returns a 429, the LLM hallucinates a coupon code that doesn't exist, MakeMyTrip renders deals via JavaScript that raw fetch can't see. A real agent must plan, act, observe, decide, and when things break, re-plan or escalate. Not crash. Not loop forever. Not silently propagate a stale deal to 40M subscribers.

Choose one of these GrabOn tasks (or propose your own of equivalent complexity):

- **Merchant Deal Audit Agent:** Given 20 GrabOn merchants (Amazon, Myntra, Zomato, Swiggy, MakeMyTrip, Nykaa, Puma, Ajio, Boat, CRED, and 10 more), crawl each deal page, extract current offers (discount %, conditions, expiry, coupon codes), compare against GrabOn's mock database, classify each as Fresh / Stale / Missing / Updated, produce a structured audit report. Must handle: Cloudflare blocks, JS-rendered content, different HTML per merchant, cached pages.
- **Transaction Reconciliation Agent:** GrabOn's deal redemptions flow through the internal tracking DB, PayU's settlement API, merchant dashboards, and Rakuten's attribution feed. Given two mock data sources (CSV + API) with 500+ transactions, identify discrepancies (missing records, value mismatches, duplicates, timezone errors), propose corrections, apply them, verify reconciliation. Must handle: IST/UTC timezone mismatches, encoding issues, partial matches.
- **Merchant Onboarding Document Agent:** New merchants submit GST certificates, PAN cards, bank proofs, signed agreements in varied formats (PDF, photos, blurry scans). Extract key fields, validate against mock verification APIs (GST portal, PAN validator), cross-reference for consistency, produce a structured onboarding record with confidence scores. Must handle: blurry scans, Hindi/Telugu text, expired documents.

Technical Requirements

- **Explicit agent loop abstraction.** Plan, Act, Observe, Decide must be recognizable phases in code. Each iteration logged: step number, phase, action, tool called, observation, decision, tokens consumed, wall-clock time.

- **At least 6 custom tools** with typed schemas, structured error responses, per-call timeouts, cost annotations. Runtime tool discovery from a registry, not hardcoded. Include one 'unreliable' tool that fails 30% of the time.
- **3 failure recovery strategies** selected by error type: (a) Retry with backoff for transient errors, (b) Re-plan with alternative tools for persistent errors (blocked? try Google cache, mobile URL), (c) Graceful degradation with partial results. A 404 is not a rate limit. The agent must distinguish.
- **Budget and safety enforcement.** Max tokens, max wall-clock time (20 merchants in 15 minutes), max tool calls, max consecutive failures. Breach = immediate halt + report of completed vs remaining.
- **Observability interface** (web UI or terminal) showing: current state per merchant, tool call history with latency and success/failure, token consumption curve, reasoning at each Decide step.
- **12+ eval scenarios:** 5 happy-path, 3 failure-and-recovery, 2 budget-exceeded, 2 impossible (agent detects and moves on, does not loop).

Minimum Bar (below this = auto-reject)

- Agent loop is a named abstraction with visible Plan/Act/Observe/Decide phases
- At least 3 custom tools with typed schemas
- At least one failure scenario where the agent recovers (not crashes)
- Budget enforcement exists and halts a runaway agent
- Eval suite exists with at least 5 automated test scenarios

What Will Be Stress-Tested in the Deep-Dive

- We will disable a tool mid-run. Does the agent re-plan?
- We will set max_tool_calls=3 on a task that needs 15. Does it halt cleanly?
- We will ask: 'At step 7, why did the agent try Google cache instead of retrying directly?'
- We will ask you to add a new tool on the spot. How long does it take?

Assignment 03: The Guard

Eval Pipeline That Guards GrabOn's AI Outputs Before They Ship

Build a production eval framework that detects quality regressions in GrabOn's AI-generated outputs across model swaps, prompt rewrites, and tool-chain changes, with statistical rigor, prompt versioning, and a CI/CD gate that blocks bad deployments

[\[Eval Framework\]](#) [\[GrabOn Outputs\]](#) [\[Regression Detection\]](#) [\[Statistical Testing\]](#) [\[CI/CD Gate\]](#) [\[Prompt Versioning\]](#)

Difficulty

Hard. 3 to 4 days.

What This Tests

Eval engineering, statistical rigor, CI integration, regression analysis

The Problem

GrabOn's agents produce outputs that people and partners actually read. Deal copy goes to 40M subscribers. GrabCredit narratives ('You qualify because your GMV grew 38% YoY...') go to Poonawalla Fincorp's compliance team. GrabInsurance recommendations determine which micro-policy a user sees at checkout. Every one of these degrades silently when a new model version ships, a prompt is tweaked, or a tool endpoint changes. A credit narrative that hallucinates transaction data is a regulatory risk. Without a rigorous eval pipeline, you catch this when a partner complains.

Technical Requirements

- **3 GrabOn-specific eval tasks** with realistic test datasets of 30+ cases each: (1) Deal copy quality across channels (email, WhatsApp at 160 chars, push at 50+100, Glance). Scoring: factual accuracy, format compliance, persuasiveness via LLM-as-judge. (2) Insurance intent classification for 30 mock deal objects across 5 categories. Scoring: accuracy, confidence calibration, edge case handling. (3) Credit narrative faithfulness for 30 mock user personas. Scoring: every claim must trace to actual data points (no hallucinated stats).
- **5 scoring functions:** (a) Factual grounding (credit narrative cites real data?), (b) Intent match (insurance recommendation matches expert label?), (c) Format compliance (channel character limits?), (d) LLM-as-judge (Opus grades another model's deal copy), (e) Semantic similarity (embedding cosine between generated and reference). Each returns 0-1 with confidence interval.
- **Statistical comparison engine.** Paired bootstrap or McNemar's test. P-values and 95% confidence intervals. 'Average went up 2%' is not a conclusion. Must handle the scenario: 'Did Telugu localization regress after upgrading from Sonnet 3.5 to Sonnet 4?'
- **GO/NO-GO gate.** GO (safe to deploy), NO-GO (regression detected), or INCONCLUSIVE. NO-GO output includes: which output type regressed, by how much, on which test cases, with what confidence.
- **Prompt versioning** integrated with git. Content hash + timestamp. On regression, diff the prompts and highlight what changed.

- **CI/CD integration.** Eval runs as a GitHub Action on every PR touching a prompt, model config, or tool schema. PR blocked on NO-GO. Provide the workflow YAML.
- **Results dashboard:** historical accuracy per output type x model x prompt version. Must answer: 'When did Telugu quality drop, and which commit caused it?'

Minimum Bar (below this = auto-reject)

- At least 2 eval tasks with 15+ test cases each
- At least 2 scoring functions that produce numerical scores, not just pass/fail
- Comparison between at least 2 models or 2 prompt versions with some form of statistical test
- GO/NO-GO gate exists and produces a clear decision
- Eval runs as a script, not manual inspection

What Will Be Stress-Tested in the Deep-Dive

- We will push a subtly bad prompt change. Does the gate catch it?
- We will swap Claude for GPT. Does the eval quantify the tradeoff per output type?
- We will ask: 'Is this 2% improvement real or noise?' Does the p-value answer it?
- We will ask you to add a new scoring function for a new output type. How fast?

Assignment 04: The Replacer

Kill a GrabOn Ops Loop. Measure the Hours. Ship the Agent.

GrabOn runs real operational workflows that consume hundreds of human-hours per month. Pick one of the two below. Build the agent that ends it. Measure the hours saved, the cost per run, and the accuracy vs human baseline.

[\[GrabOn Ops\]](#) [\[MCP Server\]](#) [\[Human-in-the-Loop\]](#) [\[ROI\]](#) [\[Deal Freshness\]](#) [\[Competitive Intel\]](#) [\[Audit Trail\]](#)

Difficulty**Hard. 3 to 4 days.****What This Tests****Process analysis, agent design for real workflows, measurable ROI, MCP spec compliance**

The Problem

AI Labs has a leaderboard: hours of manual work eliminated by agents, per quarter. This assignment is your audition entry. Choose one of two real GrabOn workflows and build the agent that replaces it.

Option A: Deal Freshness + Merchant Health Agent

GrabOn's content ops team manually checks 50+ merchant websites daily to verify deals are still live, find new deals, and flag stale listings. With 3,500+ active merchants across Fashion (24% of GMV), Travel (17%), Food (16%), Electronics (10%), and Health (8%), the team is permanently underwater. A 'Flat 40% off on Myntra' that expired two days ago but still shows on GrabOn is a broken promise to 40M subscribers and a broken data point in every downstream system.

The agent must:

- **Crawl and extract** for at least 15 real merchants (Amazon, Myntra, Zomato, Swiggy, MakeMyTrip, Nykaa, Puma, Ajio, Boat, CRED, Flipkart, BigBasket, PharmEasy, Mamaearth, Lenskart). Extract: discount %, type (flat/percentage/cashback/BOGO), conditions, min order, expiry, coupon codes. Each merchant has different HTML. No universal parser.
- **Compare and classify** against GrabOn's mock database (200+ deals). Categories: Fresh, Stale, Missing, Updated.
- **Compute Merchant Health Score** (0-100) per merchant. Below 60 = Priority Update. Rank all 15 merchants. This is the content ops lead's morning priority list.
- **Generate updated deal copy** for Stale/Missing deals matching GrabOn's format. Every number must trace to extracted data. No hallucinated discount values.
- **Handle the real-world mess:** Cloudflare, JS-rendered deals, rate limiting, buried navigation, redesigned pages.

Option B: Competitive Intelligence + Defection Early Warning Agent

GrabOn's merchant ops team spends 6+ hours weekly comparing deal listings against CouponDunia, CashKaro, and Magicpin. Fashion, Travel, and Food are 57% of GMV. If CashKaro secures an exclusive Myntra deal, the merchant ops team needs to know immediately, not on Friday.

The agent must:

- **Monitor 10 merchants across 3 competitor platforms.** Extract: discount %, deal type, exclusivity flags, last-updated timestamps, coupon codes.
- **Classify 3 types of competitive gaps:** Pricing Gap (higher discount on competitor), Exclusivity Gap (deal only on competitor), Freshness Gap (competitor updated >48h more recently).
- **Compute Defection Risk Score** per merchant (0-100). Factor: gap count, severity, trend direction, category weight (Fashion at 24% > Health at 8%). Above 70 = Priority Re-negotiation.
- **Generate a re-negotiation strategy** per Priority merchant: what to offer, what the competitor advantage is, urgency level. One paragraph the ops manager reads before calling.
- **Fire alerts:** Slack webhook + email when Fashion/Travel/Food merchants cross threshold. Include: merchant name, gap type, competitor, specific deal, recommended action. Retry on failures.
- **Handle anti-scraping:** User-Agent rotation, adaptive delays, fallback to cached versions, structured logging of failed crawls.

Technical Requirements (Both Options)

- **MCP server** following Anthropic's specification. Connectable to Claude Desktop. Demo via natural language: 'Check deal freshness for Myntra and Zomato.'
- **Confidence-based escalation.** Above 0.85 = auto-actioned. Below = human review queue with agent's reasoning. Target: >70% autonomous rate.
- **Human-in-the-loop interface.** Reviewer sees agent's work, approves/rejects/edits. Track agreement rate over time.
- **Full audit trail.** Every step logged with timestamp, merchant, tools, LLM responses, confidence, decisions. Reconstructable per merchant per run.
- **Multi-model routing.** Cheap model for HTML parsing, format validation, data diffing. Capable model for copy generation, strategy recommendations, ambiguous classification. Do not use Opus to decide if a page loaded.
- **ROI with GrabOn-scale math.** Per-merchant: human time (before), agent time, token cost, accuracy. Extrapolate: 'At 3,500 merchants, saves X hours/week at Rs. Y total weekly cost. Handles W% autonomously. Break-even vs one content ops hire: N weeks.'

Minimum Bar (below this = auto-reject)

- MCP server runs and is connectable to Claude Desktop
- At least 5 merchants processed end-to-end with structured output
- At least one crawl failure handled gracefully (not crashed)
- Confidence-based escalation exists (not everything auto-approved)
- ROI calculation exists with real numbers, not placeholders

What Will Be Stress-Tested in the Deep-Dive

- We will point the agent at a merchant it hasn't seen. Handle or escalate?
- We will check: does the agent use Sonnet to check if a page returned 200? Cost bug.
- We will verify ROI: 'At 3,500 merchants, this costs Rs. ____/month.' Credible?
- For Option B: we will manually verify competitive gaps against live competitor sites.

Assignment 05: The Coder

Coding Agent with Retrieval, Self-Testing, and Iterative Repair

Build a coding agent that takes a natural-language task, retrieves relevant context from a real codebase, generates a solution, runs tests, analyzes failures, and iterates until tests pass or explains why it cannot. Multi-model routing for different stages.

[\[Coding Agent\]](#) [\[RAG\]](#) [\[Self-Testing\]](#) [\[Iterative Repair\]](#) [\[Multi-Model\]](#) [\[Eval Benchmark\]](#)

Difficulty

Hard. 3 to 4 days.

What This Tests

Retrieval design, code generation loops, self-verification, multi-model cost optimization

The Problem

AI Labs ships code every week: agent infrastructure, MCP servers, API integrations with PayU and Poonawalla sandboxes, internal data pipelines. The gap between 'LLM can write code' and 'LLM can ship code into our codebase' is enormous. The LLM doesn't know our conventions, utility functions, test patterns, or deployment constraints. A production coding agent must understand the codebase (retrieval), generate code that fits (context-aware), verify it works (self-testing), and fix what's broken (iterative repair).

Technical Requirements

- **Real open-source codebase** with 50+ files and an existing test suite. Not a toy project you built for this. A real Python or TypeScript library, framework plugin, or CLI tool.
- **Codebase indexing and retrieval.** Parse into chunks, embed, store in vector DB. Retrieve: relevant source, imports, test files, README conventions. Demonstrably better than 'shove whole repo in context.' Measure recall on 10 queries.
- **Generate-test-fix loop:** (1) Receive task, (2) retrieve context, (3) generate code, (4) run linter + type checker + tests, (5) on failure: analyze error, retrieve more context, regenerate. Max 5 iterations. After 5: report what failed and why.
- **Multi-model routing.** Cheap model for context ranking, error parsing, test analysis. Capable model for code generation, refactoring. Track cost per task per stage.
- **3 verification layers:** (a) Static analysis (lint, types), (b) Test execution (existing + agent-written), (c) LLM-as-reviewer (separate model checks for bugs, security, conventions). All three must pass before marking done.
- **Task queue interface.** Submit multiple tasks, see status, view diffs, see verification results, cost, time per task.
- **Eval benchmark: 10 coding tasks** of varying difficulty. Report: pass rate, iterations, cost, time. Compare at least 2 model combinations.

Minimum Bar (below this = auto-reject)

- Runs against a real open-source codebase, not a toy project
- Retrieval system exists and returns relevant context (not random files)
- The generate-test-fix loop iterates at least once (not one-shot generation only)
- At least 5 of the 10 benchmark tasks attempted with results reported
- Cost tracked per task

What Will Be Stress-Tested in the Deep-Dive

- We will submit a task requiring 3+ files. Does retrieval handle it?
- We will submit an impossible task (contradicts existing tests). Does the agent detect it?
- We will check: is Opus used for error message parsing? Cost bug.
- We will compare output against a human solution.

Assignment 06: The Swarm

Multi-Agent System with Coordination, Shared State & Conflict Resolution

Build a system where 3+ specialized agents collaborate on a complex task with explicit communication protocols, shared state management, conflict resolution, and an orchestrator that enforces dependencies and budgets

[Multi-Agent] [Orchestrator] [Shared State] [Typed Messages] [Conflict Resolution] [Budget Management]

Difficulty

Hard. 3 to 4 days.

What This Tests

Multi-agent system design, coordination protocols, conflict handling, orchestration

The Problem

Single-agent systems hit a ceiling. GrabOn's competitive intelligence workflow requires web scraping skill, analytical reasoning, strategic writing, and notification formatting. One model cannot be great at all four simultaneously. Multi-agent systems decompose the problem but introduce harder challenges: communication protocols, conflict resolution, shared state, budget allocation, and debugging multi-agent conversations that go sideways.

Choose one scenario (or propose your own):

- **GrabOn Competitive Intelligence Pipeline:** Crawler Agent scrapes 10 merchants across 3 competitors (CouponDunia, CashKaro, Magicpin) on Haiku. Analyst Agent classifies gaps and computes defection risk on Sonnet. Strategist Agent writes re-negotiation briefs and threat report on Opus. Alerter Agent fires Slack/email alerts on Haiku, but only when Analyst and Strategist agree on severity. Conflict: Analyst flags a gap the Crawler's noisy extraction doesn't support. Strategist disagrees with Analyst's risk tier (seasonal, not structural). Resolve it.
- **GrabOn Merchant Underwriting Pipeline:** Data Agent pulls transaction history (deterministic, no LLM). Credit Agent produces GrabCredit assessment with explainable rationale. Insurance Agent produces GrabInsurance assessment from the same data. Compliance Agent reviews both for regulatory red flags and can veto with specific objections.
- **Incident Response System:** Monitor detects anomalies, Diagnoser identifies root cause, Responder generates and executes a fix, Communicator drafts status updates. Communicator cannot say 'fix deployed' until Responder confirms.

Technical Requirements

- **3+ specialized agents** with distinct roles, system prompts, models (not all Sonnet), and tool sets. Genuinely different capabilities.
- **Shared state store** with optimistic locking or version vectors. Every mutation attributed. Inspectable at any point.

- **Typed communication protocol.** Message types: Request, Response, Escalation, Veto, Approval, RevisionNeeded. Typed payloads, not string fields. Invalid messages rejected.
- **Conflict resolution.** Priority hierarchy, evidence-based re-evaluation, or human tiebreaker. Explicit, configurable, auditable. 'Last write wins' is not resolution.
- **Orchestrator:** enforces execution order, handles timeouts (stall >60s = intervene), enforces total budget across agents, tracks critical path.
- **At least one step that does NOT use an LLM.** Data validation, deduplication, numerical scoring, sorting. Justify why. The JD says: 'the taste to know when not to use an LLM at all.'
- **Observability timeline:** which agent is active, messages exchanged, conflicts and resolutions, state evolution, per-agent cost.
- **5 end-to-end scenarios:** 2 smooth, 2 conflict-and-resolution, 1 budget-exceeded with best partial output.

Minimum Bar (below this = auto-reject)

- At least 3 agents with different models and different tool sets
- Communication uses structured messages, not free-text between agents
- At least one conflict scenario that is resolved programmatically
- Shared state exists and is inspectable
- At least one pipeline step uses deterministic code instead of an LLM

What Will Be Stress-Tested in the Deep-Dive

- We will set budget for only 3 of 4 agents. Does the orchestrator handle it?
- We will inject wrong output from one agent. Does downstream catch it?
- We will ask: 'Why is the Data Agent not using an LLM?' Is the answer principled?
- We will read the message log. Structured and typed, or free-text soup?

Evaluation Rubric

All submissions scored 1-5 on five dimensions. No 'UI polish' dimension. This is an agent engineering role.

Dimension	5/5 Looks Like	3/5 Looks Like	Wt
Agent Architecture	Loop is a clean named abstraction. Plan/Act/Observe/Decide visible. Tool registry discoverable. State management versioned.	Loop exists but phases are muddled. Tools are hardcoded. State is ad-hoc.	25%
Eval Rigor	Eval runs, has 30+ cases, catches a real regression. Statistical comparison with p-values. CI/CD gate blocks bad changes.	Eval exists but is thin (under 10 cases). No statistical testing. Manual inspection.	20%
Failure Recovery & Production	Agent distinguishes error types. Re-plans on persistent failure. Budget kills runaway. Observability shows every decision.	Retries exist but are blind. Budget enforcement missing. Logging is print statements.	20%
Multi-LLM & Cost	4+ providers used deliberately. Cost tracked per task type. Cheaper model used for subtasks with justification. Shadow testing present.	2 providers used. Cost tracked at run-level only. No per-task breakdown.	15%
Code & README	README is an architecture doc. Diagram, tradeoffs, 'what broke first', 'what I'd change.' Code clean and reviewed.	README is install instructions only. Code works but has dead functions and hallucinated imports.	20%

Resources & API Access

Every LLM provider listed below offers a free tier or free credits on signup. This challenge is designed to be completable without spending money. If you build with LLMs professionally, you likely already have accounts on most of these. If not, signing up is part of the job.

LLM Providers (all have free tiers)

- **Anthropic (Claude Sonnet, Haiku):** Free tier on console.anthropic.com. Sufficient for development, eval runs, and the final demo.
- **OpenAI (GPT-4o, 4o-mini):** Free credits on signup at platform.openai.com. 4o-mini at \$0.15/1M input tokens costs effectively nothing for this challenge.
- **Google (Gemini Flash, Pro):** Free API access via aistudio.google.com. Generous rate limits. No credit card required.
- **Local/open models (4th provider):** Install Ollama (ollama.com), pull Llama 3 8B, Mistral 7B, or DeepSeek Coder. Free. Runs locally. Counts as a full provider.
- **Other free options:** Groq (groq.com) for fast Llama/Mistral inference. DeepSeek (platform.deepseek.com) with free credits. Together.ai and Fireworks.ai also have free tiers.

Mock Fallback Rules

If you exhaust free credits or a provider is rate-limited, you may mock responses. Rules:

- Mocks must return realistically shaped responses, not hardcoded strings.
- At least 2 of your providers must make live API calls in the final demo.
- README must clearly document which calls are live and which are mocked.
- Eval harness must run against at least 2 real providers.

Sandbox & External APIs

- **Slack, SendGrid, WhatsApp:** Use free sandbox endpoints. If a sandbox requires paid registration, mock it but build the real integration architecture (typed schemas, retry logic, error handling). Document what is live and what is mocked.
- **Web scraping:** Free. You are scraping public deal pages. Use 1-2 second delays, rotate User-Agents, handle blocks gracefully.

Submission Requirements

Send all of the following to careers@grabon.in with subject: AI Labs - [Your Name] - Assignment [Number]

- **Public GitHub repository.** Clear folder structure. We must go from git clone to running agent in under 15 minutes. If setup takes longer, that counts against you. If your current employer's IP policy prevents a public repo, email us first and we will provide a private alternative.

- **README.md** that includes: (a) What you built and why you chose this assignment, (b) Architecture diagram (hand-drawn is fine, clarity matters more than polish), (c) Per-module design decisions and tradeoffs, (d) How to run (every dependency, every env variable, every command), (e) Eval results (pass rate, accuracy, cost, latency), (f) 'What broke first' section (the hardest bug you hit and how you fixed it), (g) 'What I would change with 2 more weeks' section.
- **Eval report:** The actual output of your eval suite. Pass/fail per test case, accuracy scores, cost per run, latency data. Not a summary. The raw report. A text file, JSON, or CSV is fine.
- **Loom video (15-20 minutes).** Walk us through: (a) the happy path end-to-end, (b) at least one failure scenario where the agent recovers, (c) the eval running and producing results, (d) the architecture and key code modules. You do not need to be on camera. Screen recording with voiceover is fine. We will watch the full video.
- **Cost data.** How much did your entire development process cost in LLM tokens? How much does one full agent run cost? How much does one eval run cost? Include real numbers. A rough estimate from your provider dashboards is fine. 'I don't know' is not.
- **Your resume** as a separate PDF.

Frequently Asked Questions

Q: Can I use a framework like LangChain or CrewAI?

Yes. Use whatever you want. But during the deep-dive, we will ask you to explain what happens under every abstraction layer. If you use LangChain's AgentExecutor and cannot explain how it decides when to stop, that is a problem. The framework is a tool, not a substitute for understanding.

Q: Do I need to build a frontend?

No. A terminal UI, a Streamlit app, a simple HTML page, or even structured log output is fine. We are evaluating your agent engineering, not your CSS. If you have a dashboard or observability interface, it needs to be functional and informative. It does not need to be beautiful.

Q: What if I cannot finish everything in 3-4 days?

Submit what you have. A thoughtful partial implementation with a strong README explaining what is done, what is not, and why, is better than a rushed complete implementation with no documentation. Make sure you clear the Minimum Bar for your chosen assignment. Everything above that earns you a higher score.

Q: Can I use a different codebase for Assignment 05?

Yes. Any real open-source codebase with 50+ files and a test suite. It does not need to be a famous project. A well-maintained library with good tests is ideal.

Q: I already have a relevant project. Can I submit that instead?

Only if it was built by you, maps clearly to one of the 6 assignments, and meets the Minimum Bar. Add a README section explaining which assignment it maps to and how it meets each technical requirement. If in doubt, email us first.

Q: Can I use GPT-4o for everything and count Ollama as the second provider?

Yes, but you will score poorly on Multi-LLM & Cost. The point is not to check a box. The point is to demonstrate that you know which model to use for which task and why. Using GPT-4o for intent classification and Ollama for error parsing shows judgment. Using GPT-4o for everything shows a lack of it.

Q: What if a model provider is down during my demo recording?

Your fallback chain should handle this. That is literally what it is for. If you demo a provider going down and your agent recovering to a different model, that is a strength, not a weakness.

Q: Is this a paid assignment?

No. This is part of the interview process. It is designed to be completable using free-tier APIs and open-source tools at zero monetary cost.

Q: Who reviews my submission?

The AI Labs lead and at least one senior engineer. For assignments involving Hindi or Telugu content, a native-language reviewer will check localization quality. For assignments involving competitive crawling, we will verify gaps against live competitor sites.

Q: What happens to my code after the process?

Your code remains yours. We do not use candidate submissions in production. If you are hired, you will build production versions from scratch with the team.

Red Flags That Will Tank a Submission

- The entire system uses one model for everything. This signals you have not compared.
 - There is no eval. 'It works when I demo it' is not an eval.
 - The agent crashes on the first failure instead of recovering.
 - The README says 'run npm start' and nothing else.
 - The agent loops forever on an edge case. Budget enforcement does not exist.
 - You used a framework for everything and cannot explain what is happening beneath it.
 - The code is AI-generated but not reviewed. Hallucinated imports, dead functions, commented-out blocks, placeholder TODOs throughout.
 - Cost data is missing. You do not know how much your agent costs per run.
 - Mocked everything without documenting it. We find out during review, not from your README.
 - The Loom video is a screen recording with no audio. Walk us through it. Explain your decisions.
-

The 'will agents reshape software' debate is over. We have decided. We need the people who can ship.

Build something that runs while you sleep. Break something interesting along the way. Tell us about both.

Good luck.